

Benchmarking AI Agents for Kaggle Competitions: A Fair Three-Way Comparison with Paired Significance Testing

Midterm Report — July 2026

Wei-Hao Huang

Advisor: Tso-Jung Yen

July 28, 2026

Abstract

AI agents that automatically solve machine-learning competitions have advanced rapidly, but the methodology for deciding whether one agent is genuinely better than another remains weak: single runs, single scores, and no error model. We benchmark three methodologically distinct agents — **my-agent**, a hybrid LLM + Auto-ML agent with a cross-competition experience library built as a Claude Code Skill; **AIDE**, an open-source LLM tree-search agent with no cross-competition memory; and the **NVIDIA Kaggle Agent**, which reproduces the highest-voted public kernel — on 18 Kaggle competitions under a mechanically enforced fairness protocol (manifest-validated data isolation, a lane mutex, instruction isolation). We introduce a paired significance test whose error scale comes from Kaggle’s own public/private test split: a win counts only if the ordering replicates across the two disjoint subsets and the gap exceeds its own movement. Under this test, my-agent wins 59% of decidable duels versus 46% (AIDE) and 45% (NVIDIA), while 26% of duels are statistically undecidable — noise that a naive “higher score wins” tally would have reported as conclusions. We further show that local cross-validation disagrees with the true leaderboard in 10 of the 13 competitions for which CV comparisons are available, that the ranking of the three agents fully reverses when the evaluation set shifts from Playground-only to a broader mix, and we identify a systematic failure mode of each method. We conclude that agent-benchmark claims must be stated jointly with the composition of the evaluation set, and must be accompanied by a noise model.

1 Introduction

Machine-learning competitions are *scorable tasks*: a submission receives a single measurable quality score on a hidden test set, and a public leaderboard records how it compares against thousands of human teams. Solving them end-to-end — data understanding, validation design, feature engineering, model selection, ensembling — has traditionally demanded expert labor and extensive trial and error. LLM-driven coding agents have recently begun to automate this loop: AIDE performs LLM-guided tree search over candidate solutions [2]; ERA couples an LLM rewriter with tree search and externally injected research ideas, outperforming strong baselines across 16 Kaggle Playground competitions [1]; and industrial agents skip search altogether by reproducing the strongest public solution to a given competition [3].

However, *evaluating competition agents is currently far less rigorous than building them*. Comparisons typically report one run and one score per competition and declare the higher number the winner. Two error sources make this fragile. Competition metrics carry test-set sampling error, and top-two gaps often sit at the fourth or fifth decimal place. Agent execution is itself stochastic — the LLM samples different code at every step. When the gap between two agents is smaller than these errors, “A beats B” is reporting noise. Nor does self-evaluation offer an escape: without external, methodologically different reference points, any assessment of one’s own agent is circular.

Here we present a controlled three-way benchmark of methodologically distinct agents: **my-agent**, our hybrid design combining LLM reasoning, Auto-ML tools, and a cross-competition experience library (Figures 1 and 2); **AIDE**, LLM tree search with no cross-competition memory; and the **NVIDIA Kaggle Agent**, which reproduces the highest-voted public kernel. Both reference methods are frozen for the duration of the study. All three agents run on 18 Kaggle competitions under a mechanically enforced fairness protocol (Section 2.3), and every submission is scored on the real leaderboard via late submission. Critically, each competition scores one submission on two disjoint test subsets — public and private — and we turn this split into a paired significance test: a win counts only if the ordering replicates across both subsets and the gap exceeds its own movement (Section 3.3).

We find that my-agent wins 59% of decidable duels versus 46% (AIDE) and 45% (NVIDIA), while 26% of duels are statistically undecidable — verdicts that a naive tally would have reported as conclusions. We further find that local cross-validation disagrees with the true leaderboard in 10 of the 13 competitions where both are available, and that the three-way ranking fully reverses when the evaluation set shifts from Playground-only to a broader mix. A ranking is a property of the method *and* the evaluation set; benchmark claims must be stated jointly with both.

2 Methods

2.1 my-agent: A Hybrid Kaggle Agent

The method is implemented as a Claude Code Skill (`kaggle-agent`). Its core design is a division of labor between LLM reasoning and Auto-ML tools: the LLM (Claude Code) handles problem understanding, validation-strategy design, creative feature engineering, result interpretation, and deciding the next step; LightGBM / XGBoost / CatBoost / Optuna — invoked by the LLM via the shell — handle model selection, hyperparameter optimization, and stacking/ensembling.

Execution proceeds in six stages: data ingestion → EDA → feature engineering → modeling → evaluation → submission, where stages four and five run in one of two optimization modes:

- **Linear iteration protocol:** propose a hypothesis, run the experiment, record the result — used for the first pass and for small competitions.
- **Tree search (preferred):** once the linear protocol has produced a baseline single model and at least one blend, the agent switches to tree search (harness v3; protocol in Figure 2). Internal evaluation shows tree search winning 9 of 10 competitions with 1 tie and 0 losses.

Two supporting mechanisms:

Cross-competition experience library (`knowledge/experience.md`): every insight must carry an evidence field (`competition`, `exp #N`, `score A → score B`); entries without a score delta are rejected. The agent queries the library before EDA and before modeling, and writes validated new insights back after experiments. This is the most fundamental design difference between my-agent and the two reference methods.

Experiment records. Every experiment’s parameters, features, split scheme, and scores are logged in structured form (`experiments.json`, `experiments_tree_v3.json`), making every number in this report traceable.

Figure 1 summarizes the architecture and how the two mechanisms connect to the search loop.

Tree-search protocol. The search harness evolved as four strict supersets: v1 introduced the minimal candidate tree (a node is a complete solution; a lineage backtracks after three children without improvement); v2 made ensembles first-class node kinds (solo/blend) with out-of-fold caching and weight search; v3 — the version used for every run in this report — added a budget phase machine, duplicate handling,

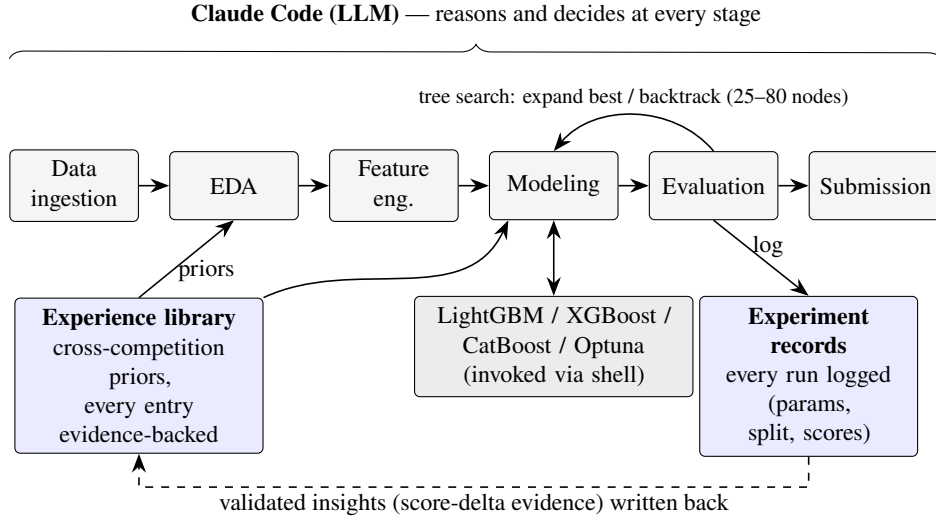


Figure 1: Architecture of my-agent, organized around its six-stage pipeline. The LLM (Claude Code) drives every stage; in the modeling–evaluation stages it iterates via tree search (25–80 CV-scored nodes per competition) and invokes Auto-ML tools through the shell. Two cross-cutting mechanisms close the loop: the experience library supplies evidence-backed priors before EDA and modeling, and every run is logged to structured experiment records, from which insights validated by a score delta are written back to the library for future competitions.

sandboxed evaluation, and resume safety; v4 adds external idea injection and is not enabled here. Figure 2 shows the v3 loop.

2.2 Two Reference Methods (Yardsticks)

AIDE (open-source agent, gen-1): LLM-driven tree search. At each step it writes a complete solution, executes it, reads the error or score, and mutates accordingly. No cross-competition memory — every competition starts from zero; the journal only persists within a single run. Executed with its default budget of 20 steps.

NVIDIA Kaggle Agent (skill): its method is “reproduce the strongest public kernel” — research the competition context, select the highest-voted solution-type kernel, port it faithfully, and retrain on local data. Its knowledge comes from publicly available solutions fetched fresh each time; nothing accumulates across competitions.

Key methodological decision. Both reference methods are frozen — no improvements of any kind. During the study we found that NVIDIA’s vote-based kernel selection is a structural weakness (Section 3.5), and the skill itself even ships a score-based selection tool, but we deliberately did not enable it. Rationale: the purpose of this study is to use them as fixed yardsticks for measuring my-agent; improving the yardsticks mid-study would make earlier and later data incomparable, and would quietly turn the benchmark into an optimization of the reference methods.

2.3 Fairness Protocol

The three agents share one machine, so the rules in Table 1 are defined and mechanically enforced.

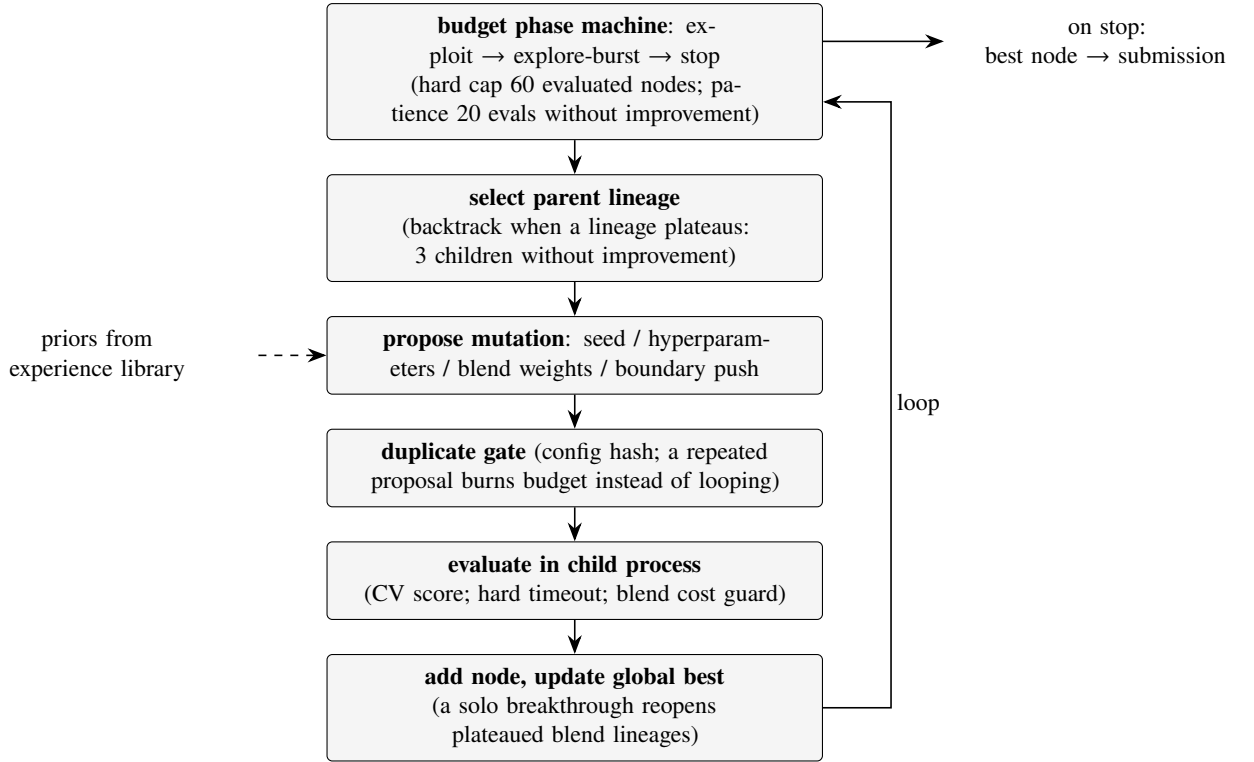


Figure 2: The v3 tree-search loop used in all my-agent runs of this report (25–80 nodes per competition). Each node is a complete, executable solution scored by local cross-validation; lineages are abandoned on plateau and the budget phase machine forces an exploration burst before stopping.

Table 1: Fairness protocol: rules and their enforcement.

Aspect	Rule	Enforcement
Data isolation	Each side reads only <code>bench-comps/<comp>/</code> , containing files validated against the official Kaggle manifest	<code>build_isolated_roots.py</code> ; officialness judged by the Kaggle API manifest, incl. members extracted from official archives
Workspace isolation	The three output directories are fully separated; each side is scored independently	Directory structure
Instruction isolation	Prompts may contain only environment facts and metric thresholds; no modeling recipes, no other side’s scores, no environment workarounds discovered by another side	Manual protocol + incident log
Resource isolation	Only one lane executes at any moment	<code>lane_lock.sh</code> mutex (atomic directory creation, reclamation on holder death, bounded waiting)
Environment defects	Always fix the environment rather than give hints (fixing removes the confound; hinting compensates only one side)	—

3 Experiments

3.1 Overview of Experimental Design

Evaluation set. 18 tabular competitions — 13 from the Kaggle Playground Series, plus afsis-soil-properties (2014, spectral regression), conway-s-reverse-game-of-life (2014, structured multi-output), cat-in-the-dat (2019, purely categorical features), tabular-playground aug-2022, and jan-2022. Every competition provides both a public and a private test-subset score — this is the inclusion criterion, because the significance test (Section 3.3) is built on the score difference between the two subsets. The three agents also ran 4 competitions for which private scores cannot be obtained (late submission closed, rolling leaderboards); these are excluded. The 18-competition set is the fixed baseline of this study.

Execution conditions. The three agents ran serially, with the mutex guaranteeing no overlap. AIDE: 20 steps (its default budget; conway stopped at 16 steps due to the 4-hour cap). NVIDIA: reproduced public kernels per its method. my-agent: the full six-stage pipeline including tree search (25–80 nodes per competition; Appendix A.6).

3.2 Experiment 1: Three-Way Capability Comparison (Raw Scores)

Goal and procedure. Measure the absolute performance of the three methods on the same set of competitions. Each side independently produces a submission, uploaded to Kaggle via late submission to obtain public and private scores, along with the percentile rank relative to all participating teams of that competition.

Table 2: Competition descriptions. Tasks fall into two families: classification (binary, ordinal; 7 competitions) and prediction/regression (incl. time series and multi-target; 11 competitions).

#	Competition	Task type	Train rows $\times$ cols
1	s3e1	Regression	37,137 $\times$ 10
2	s3e3	Classification (binary)	1,677 $\times$ 35
3	s3e5	Classification (ordinal)	2,056 $\times$ 13
4	s3e7	Classification (binary)	42,100 $\times$ 19
5	s3e9	Regression	5,407 $\times$ 10
6	s3e11	Regression	360,336 $\times$ 17
7	s3e14	Regression	15,289 $\times$ 18
8	s3e16	Regression	74,051 $\times$ 10
9	s3e19	Regression (time series)	136,950 $\times$ 6
10	s4e11	Classification (binary)	140,700 $\times$ 20
11	s5e10	Regression	517,754 $\times$ 14
12	s6e1	Regression	630,000 $\times$ 13
13	s6e2	Classification (binary)	630,000 $\times$ 15
14	afsis-soil-properties	Regression (multi-target)	1,157 $\times$ 3,600
15	cat-in-the-dat	Classification (binary)	300,000 $\times$ 25
16	conway-s-reverse-game-of-life	Structured multi-output	50,000 $\times$ 802
17	tps-aug-2022	Classification (binary)	26,570 $\times$ 26
18	tps-jan-2022	Regression (time series)	26,298 $\times$ 6

PR (percentile rank) is the submission’s percentile among all teams that participated in that competition; higher means more teams beaten. This column provides an *absolute* capability reference — it answers “where would this agent have placed had it entered the competition,” which is a separate question from the relative ordering among the three. For example, on s3e19 the three PRs are only 28.5–41.4,

Table 3: Three-way performance, public leaderboard. ↓ = lower is better, ↑ = higher is better.

#	Competition	Metric	my-agent	NVIDIA	AIDE
1	s3e1	RMSE ↓	0.56083	0.55332	0.55899
2	s3e3	ROC-AUC ↑	0.89262	0.93526	0.88484
3	s3e5	QWK ↑	0.57921	0.57515	0.59362
4	s3e7	ROC-AUC ↑	0.91116	0.91230	0.90934
5	s3e9	RMSE ↓	11.84951	11.83399	11.86657
6	s3e11	RMSLE ↓	0.29534	0.29559	0.29389
7	s3e14	MAE ↓	341.26711	338.36524	342.44918
8	s3e16	MAE ↓	1.34315	1.33566	1.33920
9	s3e19	SMAPE ↓	50.22497	47.52145	48.39715
10	s4e11	Accuracy ↑	0.94125	0.94136	0.94184
11	s5e10	RMSE ↓	0.05551	0.05562	0.05555
12	s6e1	Error ↓	8.69338	8.70480	8.69449
13	s6e2	ROC-AUC ↑	0.95367	0.95348	0.95364
14	afsis	MCRMSE ↓	0.45207	0.61276	0.45636
15	cat-in-the-dat	ROC-AUC ↑	0.80801	0.77641	0.80790
16	conway	MAE ↓	0.10771	0.11094	0.10960
17	tps-aug-2022	ROC-AUC ↑	0.58559	0.57920	0.58479
18	tps-jan-2022	SMAPE ↓	4.90883	4.43958	10.11598

Table 4: Three-way performance, private leaderboard; percentile rank (PR) in parentheses. In the verdict column, M = my-agent, N = NVIDIA, A = AIDE; “>” means the gap passes the paired significance test of Section 3.3, “≈” means indistinguishable within error.

#	Competition	Metric	my-agent (PR)	NVIDIA (PR)	AIDE (PR)	Paired verdict
1	s3e1	RMSE ↓	0.55987 (63.3)	0.55550 (93.2)	0.55552 (81.0)	$N \approx A > M$
2	s3e3	ROC-AUC ↑	0.87303 (29.4)	0.89537 (66.8)	0.86863 (26.4)	$N > M > A$
3	s3e5	QWK ↑	0.59743 (78.8)	0.56974 (75.7)	0.58138 (79.8)	$M \approx A > N$
4	s3e7	ROC-AUC ↑	0.90257 (74.3)	0.90458 (78.1)	0.90109 (67.1)	$N > M > A$
5	s3e9	RMSE ↓	12.29871 (50.6)	12.22913 (59.1)	12.30667 (44.5)	$N > M \approx A$
6	s3e11	RMSLE ↓	0.29597 (67.4)	0.29624 (66.6)	0.29445 (73.3)	$A > M > N$
7	s3e14	MAE ↓	332.43560 (76.1)	330.71616 (89.2)	332.31556 (71.2)	$N \approx A \approx M$
8	s3e16	MAE ↓	1.33859 (84.2)	1.34001 (96.4)	1.34224 (91.4)	$M \approx N > A$
9	s3e19	SMAPE ↓	50.45537 (28.5)	48.26558 (41.4)	48.34131 (40.1)	$N \approx A > M$
10	s4e11	Accuracy ↑	0.94043 (50.7)	0.94076 (52.8)	0.94056 (62.1)	$N \approx A \approx M$
11	s5e10	RMSE ↓	0.05576 (85.7)	0.05588 (59.4)	0.05579 (78.9)	$M > A > N$
12	s6e1	Error ↓	8.71733 (75.2)	8.73239 (69.5)	8.72191 (74.8)	$M > A > N$
13	s6e2	ROC-AUC ↑	0.95516 (78.8)	0.95508 (63.0)	0.95516 (76.5)	$M \approx A \approx N$
14	afsis	MCRMSE ↓	0.49517 (46.6)	0.69855 (18.8)	0.49861 (45.6)	$M > A > N$
15	cat-in-the-dat	ROC-AUC ↑	0.80241 (73.1)	0.77084 (32.6)	0.80217 (70.3)	$M > A > N$
16	conway	MAE ↓	0.10875 (98.6)	0.11189 (96.5)	0.11065 (97.9)	$M > A > N$
17	tps-aug-2022	ROC-AUC ↑	0.59055 (66.0)	0.58730 (43.0)	0.59106 (60.5)	$M \approx A > N$
18	tps-jan-2022	SMAPE ↓	6.46156 (71.5)	4.63651 (76.4)	10.66638 (28.9)	$N > M > A$
Median PR (range)			72.3 (28.5–98.6)	66.7 (18.8–96.5)	70.8 (26.4–97.9)	

showing all three fell far behind typical participants; conversely on s3e16 the PRs are 84.2–96.4 — all three are in the top tier, and their small mutual gaps carry little practical meaning there.

Public and private scores are shown in separate tables precisely because the paired test is built on their difference — the score gap of one submission across two disjoint subsets is that competition’s sampling noise.

Under a naive “bigger number wins” count, the tally is roughly my-agent 7, NVIDIA 7, AIDE 4. But that count cannot be trusted, as the next section shows.

3.3 Experiment 2: Paired Significance Testing

Why. In Table 4, several top-two gaps sit at the fourth or fifth decimal place (s5e10: 0.00003; s6e2: 0.00000). Do such gaps exceed measurement error? If not, the “verdict” is noise.

How — the error scale comes from the data itself, not from assumptions. Kaggle randomly splits each test set into two disjoint subsets, public and private, and one submission receives one score on each. These are two independent estimates of the same model, so their difference is that competition’s metric sampling noise.

A failed first attempt, worth recording. We initially used a single agent’s own public–private gap as the threshold, and 8 of 10 competitions came out as ties. The reason is that this gap is mostly *shared* across the three agents — which rows land in which subset shifts everyone together. On s5e10 the three drifts were +0.00025 / +0.00026 / +0.00024: a common shift of 0.00025 with mutual scatter of only 0.00002. Using the common shift as the threshold inflates the true noise more than tenfold — it measures a paired question with an unpaired scale.

The correct construction. Measure how much the *gap between two agents* moves across the two subsets — the common drift cancels in the subtraction. Writing d_{pub} and d_{priv} for the between-agent score difference on the public and private subsets, a verdict requires both:

$$\text{sign}(d_{\text{priv}}) = \text{sign}(d_{\text{pub}}) \quad (\text{the ordering replicates on disjoint samples}) \quad (1)$$

$$|d_{\text{priv}}| > |d_{\text{priv}} - d_{\text{pub}}| \quad (\text{the gap exceeds its own movement}) \quad (2)$$

Table 5: Empirical evidence for common drift (excerpt). Drift = private minus public score of the same submission. In seven competitions the mutual scatter is less than half the common shift.

Competition	my-agent	NVIDIA	AIDE	Common shift	Mutual scatter
s5e10	+0.00025	+0.00026	+0.00024	+0.00025	0.00002
s6e2	+0.00149	+0.00160	+0.00152	+0.00154	0.00011
s3e11	+0.00063	+0.00065	+0.00056	+0.00061	0.00009
s3e7	−0.00859	−0.00772	−0.00825	−0.00819	0.00087

Table 6: Paired-test results: 18 competitions × 3 pairings = 54 duels, of which 40 are decidable and 14 (26%) are ties.

Method	Wins	Losses	Win rate (excl. ties)
my-agent	16	11	59%
AIDE	11	13	46%
NVIDIA	13	16	45%

Interpretation. my-agent leads; AIDE and NVIDIA are hard to separate within error. A quarter of all duels cannot be decided under this test — had we reported raw counts without this layer, that fraction of the “conclusion” would have been noise. Notably, the ranking is sensitive to sample composition: restricted to the Playground subset (13 of the 18) it is NVIDIA 58% / AIDE 47% / my-agent 44%, while the full 18-competition baseline reverses it completely (Section 4.3).

Known limitation. The test covers only test-set sampling noise, not between-run agent variance (AIDE resamples code from the LLM at every step; that term should be larger). The upcoming repeatability experiment (Section 4.5) measures exactly this.

3.4 Experiment 3: Local CV and the Leaderboard Disagree

Why. The attached per-competition specification reports (Appendix A) compare the three agents on local CV, while Experiments 1–2 use the actual leaderboard. If the two agree, CV is a reliable proxy and agent evaluation need not pay the cost of real submissions; if they disagree, every CV-based agent benchmark needs re-examination. The comparison is free — both datasets already exist.

Result: 10 of 13 competitions disagree (the comparison is limited to the 13 Playground competitions covered by the spec reports). The most extreme case is s3e19 (Table 7): the direction is fully reversed. In the same competition, CV says my-agent leads by 3.7 points; the leaderboard says it trails by 2.2. Other disagreement patterns include: CV declares a tie while the LB separates the sides (s3e7, s3e9, s3e11, s5e10, s6e1), and CV declares a winner while the LB declares a tie (s3e14, s3e16).

Table 7: The most extreme CV–LB disagreement, s3e19 (SMAPE, lower is better).

Basis	my-agent	NVIDIA	Verdict
Local CV (TimeSeriesSplit, same split for both)	10.02	13.72	my-agent by a wide margin
Private leaderboard	50.46	48.27	NVIDIA wins

Why this happens. CV and LB measure different things: CV asks “how well do we do on resamples of the training data,” LB asks “how well do we do on a batch of never-seen data.” The two correlate strongly when distributions are stable, and decouple as soon as the test–train relationship changes. s3e19 is the extreme case — all three agents’ CVs sit at 10–14 while their LB scores sit at 48–50: everyone misfires together, and the magnitude of the misfire is unrelated to the CV ordering. That competition’s leaderboard is bimodal (of 1,174 teams, 265 score below 10 and 570 land in 40–60); all three agents fell into the lower mode, which CV had no way to predict.

Methodological implication. This is direct evidence for this study’s claim that agent benchmarks must submit to the real leaderboard. Comparing agents by CV is cheap and convenient, but in 10 of 13 competitions it yields conclusions that differ from real performance. The attached per-competition reports are therefore explicitly labeled CV-only, and their conclusions must not be mixed with the leaderboard verdicts of Table 4 — the two are not two answers to one question but answers to two different questions.

3.5 Experiment 4: Failure-Mode Analysis

Score tables cannot answer “why.” Reading the three agents’ execution logs and champion code competition by competition reveals systematic method-level weaknesses — worth more to agent design than any ranking.

3.5.1 AIDE: Can Diagnose, but No Mechanism Lets the Diagnosis Veto the Score

s3e16. At steps 16 and 18 AIDE ran nested CV on its own initiative, measured calibrated values of 1.3396 / 1.3412, and wrote in its own report that “the in-sample number is optimistic” — yet still selected the in-sample node. The private LB of 1.34224 confirms that the honest estimate it discarded was the correct one.

s3e19. Worse. The metric trajectory jumps from 18.42 to 4.41 within a single step:

```
step 0-10 : 18.42 ... 22.20 ... 18.42    (time-based validation)
step 11   : 4.41                        <- switched to KFold(shuffle=True)
step 12-19: 4.40, 4.37, 4.40, 4.49      (nine steps polishing a leaked number)
```

Step 11’s plan was merely to fix a LightGBM API compatibility issue; while rewriting the training loop it incidentally replaced the validation split with shuffled KFold. The SMAPE definition is identical before and after. This study confirmed the effect with a controlled experiment (same features, same model, split changed only): `KFold(shuffle=True)` yields SMAPE 4.56, while training on 2017–2020 and validating on 2021 yields 20.41 — a 4.5× gap. The competition’s test set is one full future year; shuffling rows lets the model interpolate within sequences it has memorized. AIDE noticed nothing, and spent its remaining nine steps tuning against a fake metric.

Contrast. my-agent also tried shuffled KFold in the same competition (SMAPE 4.2814) but explicitly tagged it in `experiments.json` as “DIAGNOSTIC: not used for submission,” its purpose being to isolate whether a score regression came from the split or from the features. Same trap — one side misused it, the other quarantined it.

3.5.2 NVIDIA: Votes Do Not Equal Quality

The method selects kernels by votes, but votes measure popularity and pedagogical value. On s3e3, a battle-tested solution with only 89 votes (`chunweishen/ps-s03e03-ensembling`) took the three-way best, while high-vote kernels are often tutorial-oriented rather than competition-oriented (in a getting-started competition outside this report, a 7,859-vote tutorial kernel left NVIDIA last of the three). Moreover, the s3e19 kernel it reproduced (`tumpanjawat/s3e19-course-eda-fe-lightgbm`) claims in its header “same idea as the 2nd-place solution”; NVIDIA faithfully reproduced its skeleton but missed the year-scaling step that decides the outcome (48.27 vs. the leaderboard top of 4.67) — nothing in the method ever checks the reproduced score against the source’s claimed rank.

3.5.3 A Gap Shared by All Three: Recognizing the Need for External Data

The s3e19 leaderboard is bimodal: of 1,174 teams, 265 score below 10 and 570 land in 40–60; the 10th percentile is 6.1 while the 25th percentile jumps to 22.8. All three agents (48.3 / 49.6 / 50.5) fell into the lower mode. The competition’s accepted solutions require joining per-country GDP data.

But this is not a capability ceiling: on the same-type task `tps-jan-2022`, my-agent proactively joined World Bank GDP-per-capita data (CV SMAPE 4.1793), and AIDE, using only official data, discovered calendar and Black-Friday features on its own (6.1551). s3e19 was a miss in that competition, not a ceiling of the task type.

4 Discussion

4.1 What This Study Has Achieved

1. **A credible three-way agent comparison:** 18 competitions meeting the significance-test criterion, each agent running its full method, yielding win rates of my-agent 59% / AIDE 46% / NVIDIA 45%, with a quarter of duels undecidable.

2. **A paired significance test suited to agent benchmarks:** the error scale is taken from Kaggle’s own public/private split, requiring no extra experiments to decide whether a win exceeds noise; the existence of common drift and its destruction of unpaired estimates were demonstrated empirically.
3. **Evidence that local CV is not a reliable proxy for the leaderboard:** conclusions disagree in 10 of 13 competitions, and on s3e19 the direction fully reverses.
4. **Identification of each method’s systematic weakness:** AIDE lacks a mechanism letting diagnoses veto scores; NVIDIA’s selection signal (votes) is decoupled from quality with no post-reproduction verification; all three lack the judgment “does this task need external data.”

4.2 Comparing the Three Methods

The differences lie not in tuning skill but in where knowledge comes from and how the next step is decided (Table 8).

Table 8: Design comparison of the three methods.

Aspect	my-agent	AIDE	NVIDIA
Knowledge source	Cross-competition experience library (accumulating)	None (from zero each time)	Public kernels fetched fresh each time
Search strategy	Six-stage pipeline + tree search (25–80 nodes)	Tree-shaped trial and error, 20 steps	No search; faithful reproduction
Basis for next step	Experience-library priors + OOF diagnostics	Previous step’s error or score	Highest-voted kernel
Validation rigor	Split chosen by task type; diagnostics tagged and excluded	No concept of split legality	Inherits the kernel’s split
External data	Proactively joined (GDP in jan-2022)	Never considered	Inherited from the kernel
Paired win rate (18 comps)	59%	46%	45%

Respective strengths. **NVIDIA’s strength is standing on shoulders, not its own reasoning.** It wins on s3e3, s3e7, s3e9, s3e14, and jan-2022 — competitions with mature, high-quality public solutions. It skips the entire exploration process and directly harvests hundreds of community person-hours, which is highly efficient whenever a good public solution exists; within the 13-competition Playground subset its win rate is actually the highest of the three (58%). **AIDE’s strength is search depth.** s3e11 is its only outright first place, and the edge came from a bug-fix node that accidentally enabled native categorical handling — massive trial and error stumbles into combinations a person would not think of. It needs no external knowledge to operate. **my-agent’s strength is validation rigor and knowledge accumulation.** It takes outright first on s5e10 and s6e1, and places high on s3e5 and s3e16. More important is the methodological layer: facing the same shuffled-KFold trap on s3e19, AIDE misused it for its official submission (4.5× optimism bias) while my-agent tagged it “DIAGNOSTIC: not used for submission” and excluded it. This distinction never shows up in a single score, yet it determines reliability on unseen tasks.

Respective limitations. **NVIDIA’s bottleneck is its selection signal.** Votes measure popularity, not quality (Section 3.5.2); it never checks the reproduced score against the source’s claimed rank, and when no public solution exists, the method has nothing to work with. **AIDE’s bottleneck is the missing mechanism to let diagnoses veto scores.** It can produce correct diagnoses — on s3e16 it ran nested CV unprompted and wrote “in-sample is optimistic” — yet still chose the optimistic node. On s3e19 its own metric improved 4× in one step without raising any alarm. It optimizes the displayed number, not true performance. **my-agent’s bottleneck is judgment at the problem-identification level.** The experience

library holds validated techniques (“use an L1 objective for MAE tasks”), but s3e19 required the upstream judgment “this task needs external data,” where the library has no leverage — and on jan-2022, my-agent joined GDP data yet still lost to the kernel NVIDIA reproduced, showing a gap between having the right idea and executing it fully. Moreover, its 59% lead is concentrated in the 5 non-Playground competitions (8 wins of 9 decidable duels in that subset); within the Playground subset it sits at 44% (Section 4.3).

Shared limitation. On s3e19 all three landed in the lower mode of the bimodal leaderboard (48–50 vs. top score 4.67), none recognizing the need for GDP data. Yet on the same-type jan-2022, my-agent did (CV SMAPE 4.1793) and AIDE reached 6.16 on official data alone — a per-competition miss, not a task-type ceiling.

4.3 A Ranking Is Not a Property of the Method, but of Method $\times$ Evaluation Set

The most important methodological lesson of this study: same agents, same test — swap the competition set and the ranking fully reverses (Table 9).

Table 9: Ranking reversal across evaluation sets.

Evaluation set	my-agent	AIDE	NVIDIA
Playground subset, 13 comps (win rate)	44% (last)	47%	58% (first)
Full baseline, 18 comps (win rate)	59% (first)	46%	45% (last)
Median PR (13 comps)	74.3	73.3	66.8
Median PR (18 comps)	72.3	70.8	66.7

The reversal mechanism is clearly identifiable and maps directly onto each method’s knowledge source. Three of the five non-Playground competitions are old or non-mainstream (afsis 2014, conway 2014, cat-in-the-dat 2019): their public kernels are either stale or tutorial-grade, so NVIDIA’s “reproduce the top-voted kernel” loses the mature solution ecosystem it enjoys in the Playground series — its PR drops to 18.8 (afsis) and 32.6 (cat-in-the-dat). my-agent takes 8 of the 9 decidable duels in these 5 competitions — not because it got stronger, but because its opponent’s precondition disappeared.

Two observations:

1. **Median PR orders the three identically on both evaluation sets** (my-agent > AIDE > NVIDIA), while the paired win rate reverses. Absolute-position metrics are more stable than relative verdicts, because the latter binarize many near-tie gaps and are therefore highly sensitive to sample composition.
2. **“Which agent is stronger” is an incomplete question.** The correct question is “stronger on what distribution of competitions”: with mature public solutions, reproduction (NVIDIA) dominates; on obscure, old, build-your-own-pipeline tasks, autonomous methods (my-agent) dominate. Any single-number benchmark ranking hides an unstated assumption about the competition distribution.

Methodological implication. An agent benchmark’s conclusion must be stated together with the composition of its evaluation set, and should report both an absolute metric (PR) and a relative one (paired win rate) — the contrast between this study’s two evaluation sets is direct evidence for that claim.

4.4 Limitations of the Experimental Design

- **The significance test covers only test-set sampling noise**, not between-run agent variance. AIDE’s term is expected to be largest (the LLM resamples code at every step); once included, some of the current 40 decisive results may revert to ties.

- **The asymmetry created by the experience library is disclosed, not eliminated.** my-agent carries cross-competition memory; the other two cold-start — a design difference rather than a flaw, but it colors the interpretation of the conclusions.
- **The sample is small, and Section 4.3 shows the ranking is sensitive to set composition.** With 18 competitions and 54 duels, the gap between 59% and 45% remains a rough estimate; adding or removing a single competition can move it by one to two percentage points.

4.5 Work in Progress and Future Directions

In progress.

1. **Repeatability experiment:** take the 5 closest-fought competitions (s6e2, s4e11, s5e10, s3e16, s3e11), re-run AIDE twice on each, measure between-run variance, and produce error bars — the direct remedy for the first limitation above.
2. **Mid-complexity competitions:** us-patent (NLP), ventilator (time series), SIIM-ISIC (imaging) — re-run all three agents with the machine fully dedicated, testing whether the conclusions extend beyond Playground-type tasks.

Future directions.

- **Extend the experience library to the problem-identification level:** it currently stores techniques (“use L1 objectives for MAE tasks”) but lacks upstream judgments such as “does this task type need external data” or “is the test set a future time window.” The s3e19 miss falls exactly in this gap.
- **Introduce validation gates:** AIDE’s two failures (choosing the optimistic node on s3e16; tuning a leaked metric on s3e19) both stem from having no mechanism that lets a diagnosis veto a score. my-agent could add a hard rule — a single-step metric improvement beyond a threshold automatically triggers a split-legality check.
- **Study the selection signal:** NVIDIA’s bottleneck (votes $\neq$ quality) is a concrete, improvable mechanism. my-agent is not bound by the yardstick freeze and can implement selection by author rank or kernel-claimed score, checking reproduced scores against source claims.
- **Measure the experience library’s actual value:** we currently know only that it creates an asymmetry, not how much it contributes. An ablation with the library disabled would measure it directly.

A Experimental Procedure and Per-Competition Specification Reports

A.1 Per-Competition ML Specification Reports (Attached)

Full specifications for every agent on every competition, written to the five sections of Tso-Jung Yen’s ML specification framework (ml_pipeline_modules) — Data / Models & Architecture / Training / Inference / Evaluation & Benchmarking — per competition: 15 reports for my-agent ([kaggle/docs/ml_specs/](https://kaggle.com/docs/ml_specs/), MD + PDF) and 17 for AIDE ([aideml-runs/docs/ml_specs_aide/](https://aideml-runs.com/docs/ml_specs_aide/), MD + PDF).

Every number in these reports is extracted deterministically by `collect.py/collect_aide_facts.py` from that competition’s structured records (`facts.json`, `eda_summary.json`, `experiments_tree_v3.json`, `journal.json`), never generated by an LLM; unavailable fields are marked “not recorded” rather than left blank or estimated. The my-agent report set additionally ships `RUBRIC.md`, an 8-item plan-alignment checklist verified per competition (all 15 pass 8/8).

This appendix does not repeat per-competition content; below are only the cross-competition rules and the execution environment that individual reports cannot cover.

A.2 Cross-Competition Rule: Split Strategy Is Determined by Task Type

This is the single highest-impact design item in the study, visible only through cross-competition comparison (Table 10).

Table 10: Validation-split policy by task type.

Task type	Required split	Rationale
Time-series extrapolation (test set later than training period)	Time-based split (<code>year < y</code> vs <code>year == y</code>) or <code>TimeSeriesSplit</code>	Each validation block must be strictly later than its training window, mirroring the true train–test gap
Text pairing (same anchor in many rows)	<code>GroupKFold</code> by anchor	Otherwise one anchor spans train and validation, and the model memorizes anchors instead of semantic relations
Generic tabular (continuous i.i.d. target)	Shuffled <code>KFold</code> / <code>StratifiedKFold</code>	No temporal or group structure

Shuffled `KFold` on time-series data is strictly forbidden. AIDE switched to shuffled `KFold` on s3e19 as a side effect of an unrelated bug fix; a controlled experiment measured a 4.5× optimism bias from that one error (same features, same model, split changed only: 4.56 vs. 20.41). my-agent tried the same split in the same competition but explicitly tagged it as diagnostic and excluded it from submission.

A.3 Cross-Competition Rule: DL-to-GBDT Field Mapping

The specification framework contains several deep-learning-specific fields; this study is GBDT-centric, and all per-competition reports follow the mapping in Table 11.

Table 11: Mapping of deep-learning framework fields to their GBDT counterparts.

Framework field	GBDT counterpart
Learning Rate	Boosting shrinkage (<code>learning_rate</code>)
Learning Rate Scheduler	N/A — convergence governed by CV early stopping, not a schedule
Batch Size	N/A — full-dataset histogram boosting, not mini-batches
Model Complexity	Expressed as “number of trees × leaves,” not dense parameter count
Transfer Learning	N/A (no pretrained weights); analogue: prior injection from the experience library
Data Augmentation	N/A (tabular); analogue: feature engineering and seed bagging

A.4 Execution Environment and Reproducibility

Table 12: Execution environment.

Item	Detail
Hardware	ARM64 Linux (Ubuntu 24.04), NVIDIA GB10 GPU
Environment management	uv + uv.lock (frozen versions); Python 3.13
Data source	bench-comps/<comp>/data/ — symlinks to official files validated against the Kaggle manifest
Resource isolation	lane_lock.sh mutex; only one lane runs at any moment
Monitoring	bench-watchdog.timer, checking every 15 minutes for (a) machine idle with work pending, (b) process alive but no output for 30 minutes, (c) individual driver death
Determinism	Fixed seeds; LightGBM with deterministic=true, force_row_wise=true, fixed num_threads — necessary for cross-process reproducibility
Per-step memory cap	32 GB (exceeding it marks the node buggy; a single step once consumed 76 GB and stalled the machine)
Output isolation	my-agent kaggle/competitions/; AIDE aideml-runs/; NVIDIA nvidia-kaggle-runs/

A.5 Silent Failures Encountered and Fixed

During the study we found five classes of failure that raise no errors; all were fixed and mechanized before the reported runs (Table 13).

Table 13: Five silent failures of the experimental design, their detection, and fixes.

#	Failure	Impact	Detection	Fix
1	Harness copied my-agent’s entire working directory to AIDE, exposing processed features and tuned hyperparameters	4 AIDE results voided	Inspecting which files the champion code actually opened	Manifest-validated isolated roots; 4 competitions re-run
2	Runs under resource contention (one lane at load 17, another with the machine to itself)	Unequal conditions	Comparing lane execution windows	lane_lock.sh mutex
3	Watchdog judged health by “machine busy overall”; a dead lane went unnoticed all night	One night of capacity lost	Per-driver check (no completion marker, no process)	Watchdog extended with per-driver checks
4	Aggregation took “best score across all experiments,” counting leakage experiments tagged as diagnostic	3 competitions’ numbers wrong	Scanning logs for exclusion tags	Fixed and disclosed
5	Headless-agent instructions enumerated six stages but omitted tree search; the agent followed the prompt, not the skill	5 competitions ran a truncated method	Checking for experiments_tree*.json	Prompt changed to “follow the skill; this prompt is not a pipeline definition”

Measured contrast for failure 5 (afsis-soil-properties): the truncated run produced 0 tree-search nodes, CV MCRMSE 0.44817, in 32 minutes; the fixed run produced 80 nodes, CV MCRMSE 0.444076, in 17 minutes. After the fix, the score improved *and* the run got faster — consistent with the skill’s internal evidence that tree search wins 9 / ties 1 / loses 0.

A.6 Tree-Search Size per Competition (my-agent)

Table 14: Tree-search node counts per competition (my-agent).

Competition	Nodes	Competition	Nodes	Competition	Nodes
s3e1	22	s3e14	62	s5e10	50
s3e3	22	s3e16	24	s6e1	48
s3e5	23	s3e19	22	s6e2	60
s3e7	63	s4e11	60	afsis	80
s3e9	26	cat-in-the-dat	65	conway	25
tps-aug-2022	60	tps-jan-2022	60		

B Code and Skill Packaging

All my-agent code and benchmark artifacts are archived in the project repository github.com/930731haohao-afk/kaggle-agent (private; access granted on request). File paths in the tables below link to the corresponding files in that repository; ai_agents/-level infrastructure scripts are mirrored under [benchmark_infra/](#), and cross-agent result artifacts under [benchmark_results/](#).

B.1 my-agent Skill Definition

Table 15: Skill definition files.

File	Purpose
kaggle/.claude/skills/kaggle-agent/SKILL.md	Main skill definition: six-stage pipeline, tree-search switching condition, experience-library query timing
.../references/01_setup.md – 06_submission.md	Detailed per-stage instructions
.../references/07_tree_search.md	Tree-search protocol (preferred optimization mode)
kaggle/tree_search/harness_v3.py	Tree-search harness
kaggle/knowledge/experience.md	Cross-competition experience library (every entry carries an evidence field)
kaggle/knowledge/idea_bank.md	Pool of unvalidated ideas

B.2 Experiment Execution Scripts

Table 16: Experiment execution scripts.

File	Purpose
<code>ai_agents/bench-comps/build_isolated_roots.py</code>	Build manifest-validated isolated data roots; officialness judged via the Kaggle API, incl. members of official archives
<code>ai_agents/lane_lock.sh</code>	Lane mutex (atomic directory creation, reclamation on holder death, bounded waiting)
<code>ai_agents/bench_watchdog.sh</code>	Dual-signal monitoring: machine level (alive + advancing) and per driver
<code>ai_agents/kaggle/run_myagent_headless.sh</code>	Unattended my-agent execution (one headless session per competition)
<code>ai_agents/aideml/run_comp.py</code>	Single-competition AIDE run (supports AIDE_COMP_ROOT)
<code>ai_agents/aideml/rerun_20steps.py</code>	Batch AIDE runs (20-step budget)
<code>ai_agents/aideml/collect_aide_facts.py</code>	Deterministic fact extraction from AIDE journals
<code>ai_agents/nvidia-kaggle-runs/run_ready_reproductions.py</code>	NVIDIA reproduction runner (only scripts passing smoke tests)

B.3 Data and Results

Table 17: Data and result artifacts.

Item	Path
Three-way score master table	<code>ai_agents/aideml-runs/three_way_scores.csv</code>
Full three-way comparison report	<code>ai_agents/aideml-runs/THREE_WAY_REPORT.md</code> (with PDF)
AIDE per-competition ML spec reports (17)	<code>ai_agents/aideml-runs/docs/ml_specs_aide/</code>
my-agent per-competition ML spec reports (15)	<code>ai_agents/kaggle/docs/ml_specs/</code>
Coverage status table	<code>ai_agents/BENCHMARK_COVERAGE.md</code>
Per-competition experiment logs	<code>kaggle/competitions/<comp>/experiments.json, experiments_tree_v3.json</code>

B.4 Minimal Steps to Reproduce This Study

```
# 1) Build isolated data roots (per official Kaggle manifest)
python ai_agents/bench-comps/build_isolated_roots.py --write <comp>

# 2) my-agent, one competition
cd ai_agents/kaggle && claude -p "<see run_myagent_headless.sh PROMPT>" \
    --dangerously-skip-permissions

# 3) AIDE, one competition (20 steps)
AIDE_COMP_ROOT=~/.ai_agents/bench-comps \
    python ai_agents/aideml/run_comp.py <comp> --shim --steps 20

# 4) NVIDIA, one competition
```

```
cd ai_agents/nvidia-kaggle-runs/<comp> && python reproduce.py
```

```
# 5) Submit and retrieve public/private scores
```

```
KAGGLE_API_TOKEN=$(cat ~/.kaggle/huang_token) \  
    kaggle competitions submit -c <comp> -f submission.csv -m "<msg>"
```

Environment: ARM64 Linux (Ubuntu 24.04), NVIDIA GB10 GPU, uv-managed Python 3.13, versions frozen in `uv.lock`.

Every number in this report is traceable to the structured records listed above. Experiments are ongoing; this report will be updated as the items in Section 4.5 complete.

References

- [1] Aygün, E., et al. An AI system to help scientists write expert-level empirical software. *Nature* (2026). <https://doi.org/10.1038/s41586-026-10658-6>
- [2] Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138 (2025).
- [3] NVIDIA. Kaggle Agent: kernel-reproduction agent skill (software), 2026.